Experiment 8

AIM: Develop Inventory Management Module using ADO.NET & CRUD Operations

Source: App_Start/RouteConfig.cs

Code:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;
using System.Web.Mvc;
using System.Web.Routing;

namespace connecting_sql
{
    public class RouteConfig
    {
        public static void RegisterRoutes(RouteCollection routes)
        {
            routes.IgnoreRoute("{resource}.axd/{*pathInfo}");

            routes.MapRoute(
                name: "Default",
                url: "{controller}/{action}/{id}",
                defaults: new { controller = "Products", action = "Index", id =
UrlParameter.Optional }
            );
        }
    }
}
```

Source: Controllers/ProductsController.cs

Code:

```
using System;
using System.Collections.Generic;
using System.Data.SqlClient;
using System.Web.Mvc;
using connecting_sql.Models;

namespace connecting_sql.Controllers
{
    public class ProductsController : Controller
    {
        string cs = "Data Source=(localdb)\\MSSQLLocalDB;Initial Catalog=TestDB;Integrated
Security=True";


        public ActionResult Index()
        {
            List<Products> productList = new List<Products>();



            using (SqlConnection con = new SqlConnection(cs))
            {
                string query = "SELECT * FROM Products";
                SqlCommand cmd = new SqlCommand(query, con);
                con.Open();
                SqlDataReader rd = cmd.ExecuteReader();

                while (rd.Read())
                {
                    productList.Add(new Products
                    {
                        Id = Convert.ToInt32(rd["ProductId"]),
                        Name = rd["ProductName"].ToString(),
                        Quantity = Convert.ToInt32(rd["Quantity"]),
                        Price = Convert.ToSingle(rd["Price"])
                    });
                }
            }

            return View("Home",productList);
        }

        public ActionResult Create()
        {
            return View();
        }

        [HttpPost]
        public ActionResult Create(Products prod)
        {
            using (SqlConnection con = new SqlConnection(cs))
            {
```

```
                string query = "INSERT INTO Products (ProductName, Quantity, Price) VALUES
(@ProductName, @Quantity, @Price)";
                SqlCommand cmd = new SqlCommand(query, con);
                cmd.Parameters.AddWithValue("@ProductName", prod.Name);
                cmd.Parameters.AddWithValue("@Quantity", prod.Quantity);
                cmd.Parameters.AddWithValue("@Price", prod.Price);

                con.Open();
                cmd.ExecuteNonQuery();
            }

            return RedirectToAction("Index");
        }

        public ActionResult Edit(int id)
        {
            Products prod = null;

            using (SqlConnection con = new SqlConnection(cs))
            {
                string query = "SELECT * FROM Products WHERE ProductId = @ProductId";
                SqlCommand cmd = new SqlCommand(query, con);
                cmd.Parameters.AddWithValue("@ProductId", id);

                con.Open();
                SqlDataReader rd = cmd.ExecuteReader();



                if (rd.Read())
                {
                    prod = new Products
                    {
                        Id = Convert.ToInt32(rd["ProductId"]),
                        Name = rd["ProductName"].ToString(),
                        Quantity = Convert.ToInt32(rd["Quantity"]),
                        Price = Convert.ToSingle(rd["Price"])
                    };
                }
            }

            if (prod == null)
            {
                return HttpNotFound();
            }

            return View(prod);
        }

        [HttpPost]
        public ActionResult Edit(Products prod)
        {
            using (SqlConnection con = new SqlConnection(cs))
            {
                string query = "UPDATE Products SET ProductName = @ProductName, Quantity =
@Quantity, Price = @Price WHERE ProductId = @ProductId";
                SqlCommand cmd = new SqlCommand(query, con);
                cmd.Parameters.AddWithValue("@ProductId", prod.Id);
```

```
                cmd.Parameters.AddWithValue("@ProductName", prod.Name);
                cmd.Parameters.AddWithValue("@Quantity", prod.Quantity);
                cmd.Parameters.AddWithValue("@Price", prod.Price);

                con.Open();
                cmd.ExecuteNonQuery();
            }

            return RedirectToAction("Index");
        }
        [HttpPost]
        public ActionResult Delete(int id)
        {
            using (SqlConnection con = new SqlConnection(cs))
            {
                string query = "DELETE FROM Products WHERE ProductId = @ProductId";
                SqlCommand cmd = new SqlCommand(query, con);
                cmd.Parameters.AddWithValue("@ProductId", id);

                con.Open();
                cmd.ExecuteNonQuery();
            }

            return RedirectToAction("Index");
        }
    }
}
```

Source: Models/Products.cs

Code:

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Web;

namespace connecting_sql.Models
{
    public class Products
    {
        public int Id { get; set; }
        public String Name { get; set; }
        public int Quantity { get; set; }
        public float Price { get; set; }
    }
}
```

Source: Views/Products/Create.cshtml

Code:

```
@model connecting_sql.Models.Products

@{
    ViewBag.Title = "Create Product";
}

<h2>Create Product</h2>

@using (Html.BeginForm())
{
    @Html.AntiForgeryToken()

    <div class="form-horizontal">
        <hr />
        @Html.ValidationSummary(true, "", new { @class = "text-danger" })

        <div class="form-group">
            @Html.LabelFor(model => model.Name, htmlAttributes: new { @class = "control-label col-
md-2" })
            <div class="col-md-10">
                @Html.EditorFor(model => model.Name, new { htmlAttributes = new { @class = "form-
control" } })
            </div>
        </div>

        <div class="form-group">
            @Html.LabelFor(model => model.Quantity, htmlAttributes: new { @class = "control-label
col-md-2" })
            <div class="col-md-10">
                @Html.EditorFor(model => model.Quantity, new { htmlAttributes = new { @class =
"form-control" } })
            </div>

        </div>

        <div class="form-group">
            @Html.LabelFor(model => model.Price, htmlAttributes: new { @class = "control-label
col-md-2" })
            <div class="col-md-10">
                @Html.EditorFor(model => model.Price, new { htmlAttributes = new { @class = "form-
control" } })
            </div>
        </div>

        <div class="form-group" style="margin-top:15px;">
            <div class="col-md-offset-2 col-md-10">
                <input type="submit" value="Create" class="btn btn-success" />
                @Html.ActionLink("Back to List", "Index", null, new { @class = "btn btn-secondary"
})
            </div>
        </div>
    </div>
}
```

Source: Views/Products/Edit.cshtml

Code:

```
@model connecting_sql.Models.Products

@{
    ViewBag.Title = "Edit Product";
}

<h2>Edit Product</h2>

@using (Html.BeginForm())
{
    @Html.AntiForgeryToken()

    <div class="form-horizontal">
        <hr />
        @Html.ValidationSummary(true, "", new { @class = "text-danger" })
        @Html.HiddenFor(model => model.Id)

        <div class="form-group">
            @Html.LabelFor(model => model.Name, htmlAttributes: new { @class = "control-label col-
md-2" })
            <div class="col-md-10">
                @Html.EditorFor(model => model.Name, new { htmlAttributes = new { @class = "form-
control" } })
            </div>
        </div>

        <div class="form-group">
            @Html.LabelFor(model => model.Quantity, htmlAttributes: new { @class = "control-label
col-md-2" })
            <div class="col-md-10">

    @Html.EditorFor(model => model.Quantity, new { htmlAttributes = new { @class = "form-control"
} })
            </div>
        </div>

        <div class="form-group">
            @Html.LabelFor(model => model.Price, htmlAttributes: new { @class = "control-label
col-md-2" })
            <div class="col-md-10">
                @Html.EditorFor(model => model.Price, new { htmlAttributes = new { @class = "form-
control" } })
            </div>
        </div>

        <div class="form-group" style="margin-top:15px;">
            <div class="col-md-offset-2 col-md-10">
                <input type="submit" value="Save" class="btn btn-primary" />
                @Html.ActionLink("Back to List", "Index", null, new { @class = "btn btn-secondary"
})
            </div>
        </div>
    </div>
}
```

Source: Views/Products/Home.cshtml

Code:

```
@model IEnumerable<connecting_sql.Models.Products>

@{
    ViewBag.Title = "Product Inventory";
}

<h2>Product Inventory</h2>

<p>
    @Html.ActionLink("Create New Product", "Create", null, new { @class = "btn btn-primary" })
</p>

<table class="table table-bordered table-striped">
    <tr>
        <th>@Html.DisplayNameFor(model => model.Name)</th>
        <th>@Html.DisplayNameFor(model => model.Quantity)</th>
        <th>@Html.DisplayNameFor(model => model.Price)</th>
        <th>Actions</th>
    </tr>

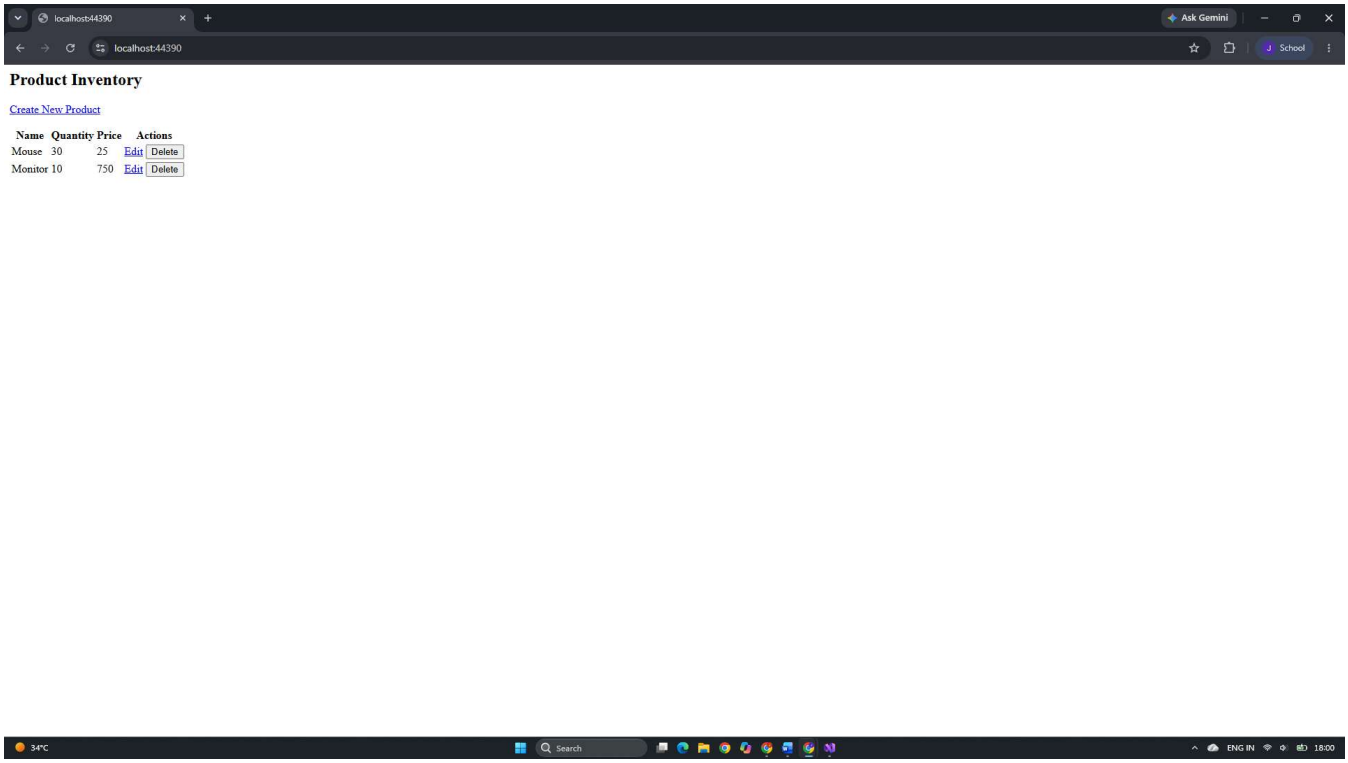
@foreach (var item in Model) {
    <tr>
        <td>@Html.DisplayFor(modelItem => item.Name)</td>
        <td>@Html.DisplayFor(modelItem => item.Quantity)</td>
        <td>@Html.DisplayFor(modelItem => item.Price)</td>
        <td>
            @Html.ActionLink("Edit", "Edit", new { id = item.Id })
            @using (Html.BeginForm("Delete", "Products", new { id = item.Id }, FormMethod.Post,
new { style = "display:inline;" }))

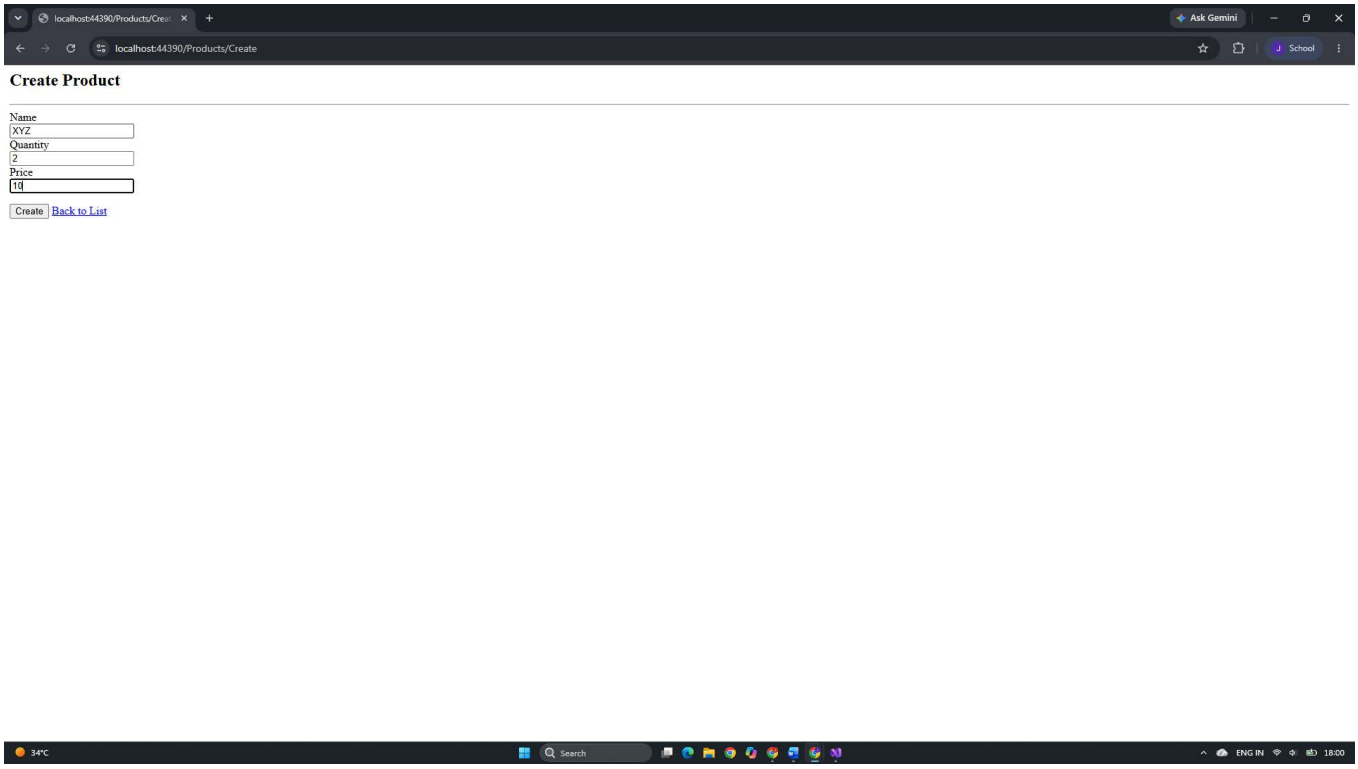


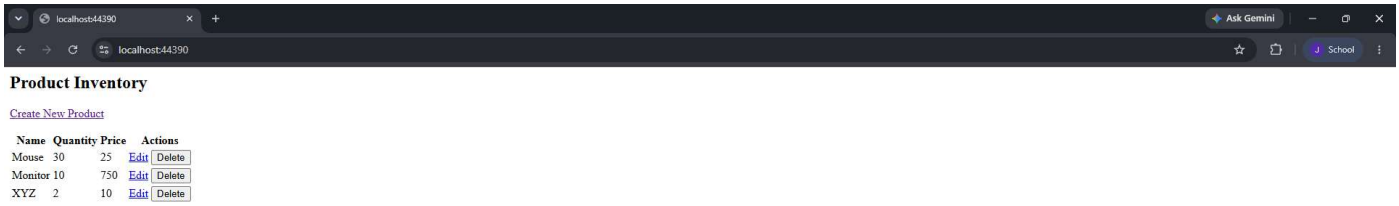
            {<button type="submit" onclick="return confirm('Are you sure you want to delete this
product?');">Delete</button>}
        </td>

    </tr>
}
</table>
```

Output:


localhost:44390
Ask Gemini
School
Product Inventory
Create New Product
Name Quantity Price Actions
Mouse 30 25 Edit Delete
Monitor 10 750 Edit Delete


localhost:44390/Products/Create
Ask Gemini
School
Create Product
Name
XYZ
Quantity
2
Price
10
Create
Back to List



Product Inventory

Create New Product

Name	Quantity	Price	Actions
Mouse	30	25	Edit Delete
Monitor	10	750	Edit Delete
XYZ	2	10	Edit Delete



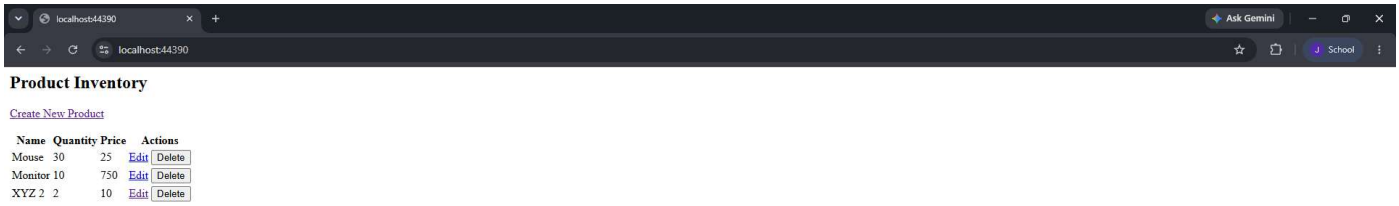


Edit Product

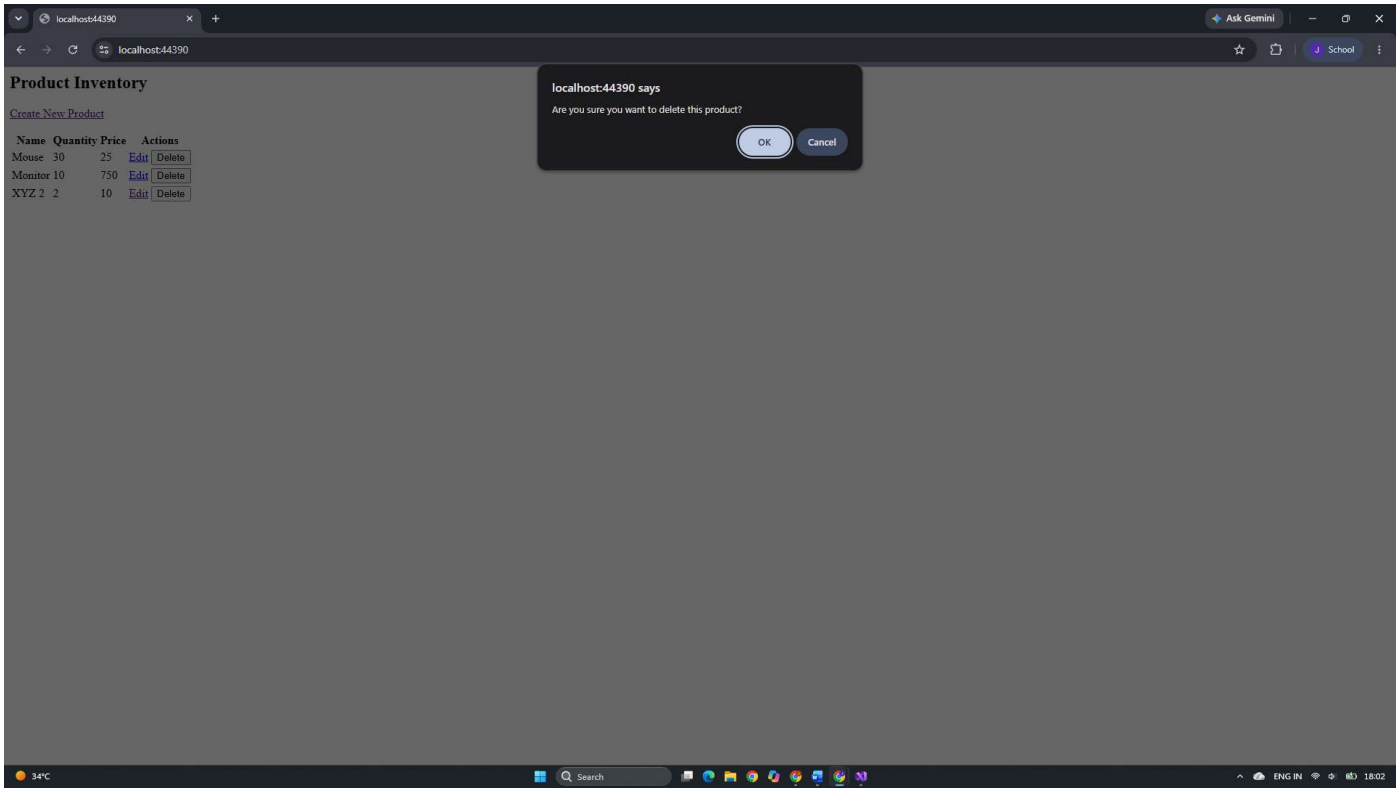
Name
XYZ 2
Quantity
2
Price
10

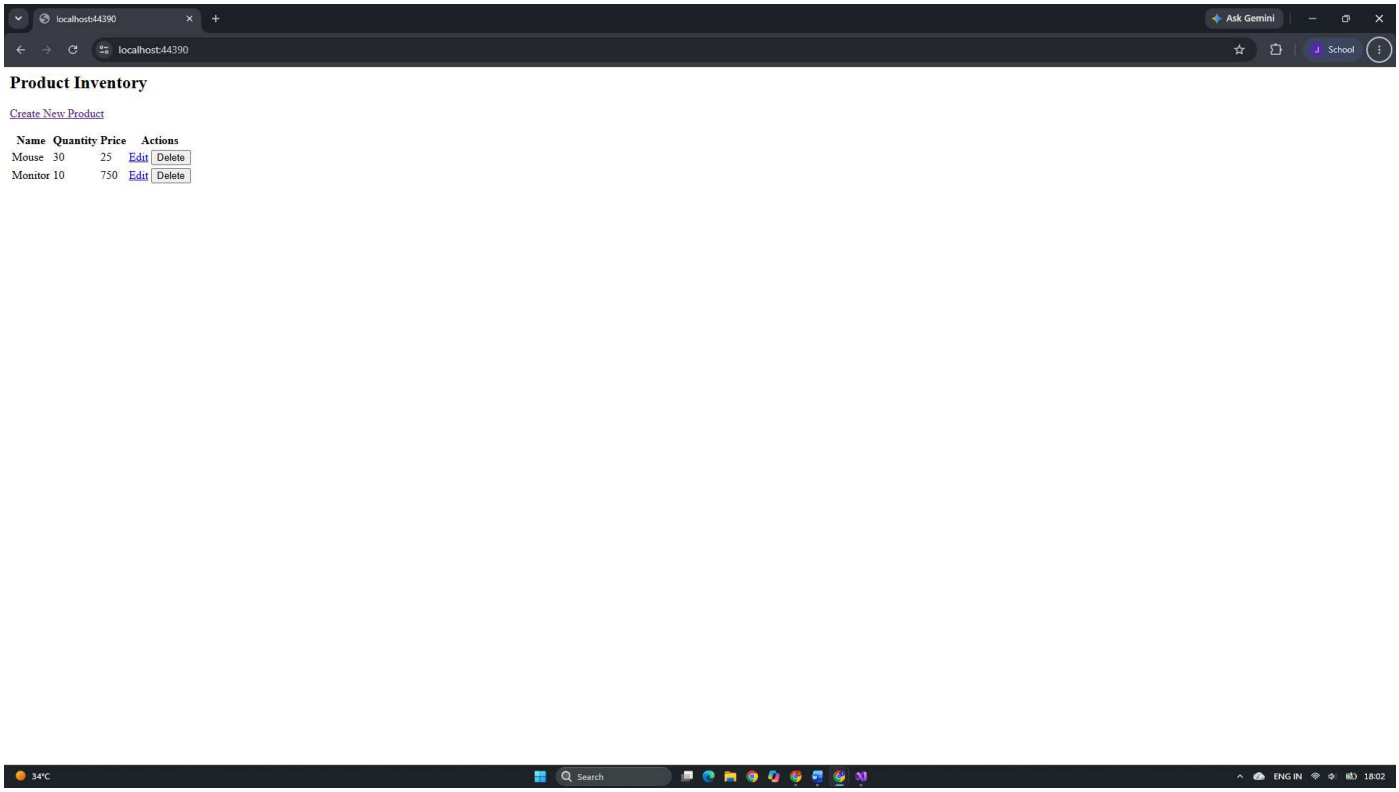
Save Back to List




localhost:44390
Ask Gemini
localhost:44390
School
Product Inventory
Create New Product
Name Quantity Price Actions
Mouse 30 25 Edit Delete
Monitor 10 750 Edit Delete
XYZ 2 2 10 Edit Delete


34°C
Search
ENG IN
18:02


localhost:44390
Ask Gemini
localhost:44390
School
Product Inventory
Create New Product
Name Quantity Price Actions
Mouse 30 25 Edit Delete
Monitor 10 750 Edit Delete
XYZ 2 2 10 Edit Delete
localhost:44390 says
Are you sure you want to delete this product?
OK
Cancel
34°C
Search
ENG IN
18:02



Github Url:
https://github.com/jd-thakrar/Uni-DotNet-Work/tree/master/connecting_sql/connecting_sql